

Database Systems Lab

Project Proposal

1. Objective of the Proposal

The purpose of this proposal is to clearly explain:

- The problem being addressed — manual and inefficient student fee management in educational institutions.
- The system to be developed — a Student Fee Management System (SFMS) that automates fee tracking, payment recording, and report generation.
- The role of the database — a relational database storing all student, fee, and payment data with full integrity and consistency.

2. Project Team Information

Student 1 Name	Muhammad Aimal
Program & Group	BS Computer Science — [A]
Student 2 Name	[Muhammad Ghani]
Project Title	Student Fee Management System (SFMS)

3. Introduction & Background

Educational institutions deal with hundreds of student fee records every semester. Managing these records manually through paper files or spreadsheets is time-consuming, error-prone, and difficult to scale. In Pakistan, many mid-sized institutions still lack a proper digital fee management system, leading to lost records, miscalculated dues, and delays in financial reporting. A dedicated software system backed by a well-designed relational database can solve these problems efficiently and reliably.

4. Problem Statement

The following real-world problems exist in manual fee management:

- Manual data entry leads to errors, duplicate records, and lost payment receipts.
- Students and administrators cannot easily check pending dues or payment history.
- Semester-end financial reporting requires hours of manual calculations.
- There is no reliable way to track overdue payments or generate summaries.

Students, parents, and finance department staff are all affected. A database solution is required because the data is inherently relational — payments must be linked to students, students to departments, and fees to semesters — making a flat file or spreadsheet approach inadequate.

5. Proposed Solution

We propose a desktop-based Student Fee Management System built using Python (Tkinter) for the interface and MySQL as the backend database. The system will allow administrators to register students, define fee structures per department and semester, record payments, generate receipts, and view financial reports. All data will be stored in a normalized relational database ensuring consistency and fast retrieval. The system replaces manual processes with reliable, database-driven operations.

6. Objectives of the System

- Maintain accurate and centralized records of all student fee data.
- Allow fast recording and retrieval of payment transactions.
- Automatically calculate pending dues based on fee structure.
- Generate payment receipts and semester-wise financial reports.
- Ensure data integrity through primary keys, foreign keys, and constraints.
- Provide a simple and usable interface for finance department staff.

7. Scope of the Project

Included in the project:

- Student registration and profile management.
- Fee structure definition per department and semester.
- Payment recording and receipt generation.
- Pending dues calculation and financial reporting.
- Search and filter functionality for students and payments.

Excluded from the project:

- Online payment gateway or mobile application interface.
- Automated SMS or email notifications.
- Multi-branch or multi-institution support.

8. Role of Database System

The database is the core component of this system. It will store: student profiles, department and program information, semester-wise fee structures, payment transaction records, and generated receipts. A relational database is necessary because all these entities are interconnected — a payment belongs to a student, and a student belongs to a department with a defined fee structure.

The system will perform full CRUD operations: inserting new students and payments, reading records for display and reports, updating fee structures, and deleting invalid entries. Data integrity will be

enforced through NOT NULL constraints, UNIQUE roll numbers, foreign key relationships, and CHECK constraints on payment amounts.

9. System Features / Functional Requirements

- Student Registration — Add, edit, and view student profiles with roll number, name, department, and semester.
- Fee Structure Management — Define total fee amounts per department and semester with due dates.
- Payment Recording — Record payments with amount, date, and payment method (cash/bank).
- Dues Calculation — Automatically compute remaining balance per student.
- Receipt Generation — Generate a printable receipt for each confirmed payment.
- Search and Filter — Search students by name or roll number; filter payments by date or status.
- Semester Report — View total fees collected and total pending dues for a selected semester.
- Department Summary — View fee collection summary grouped by department.

10. Preliminary Data Design

Main entities and their relationships:

Entity	Key Attributes	Relationship
Student	student_id, name, roll_no, semester	Belongs to one Department
Department	dept_id, dept_name, program	Has many Students
Fee Structure	fee_id, dept_id, semester, total_amount, due_date	Linked to one Department
Payment	payment_id, student_id, amount, date, method	Made by one Student
Receipt	receipt_id, payment_id, generated_date	Generated for one Payment

11. Existing Systems / Comparative Analysis

Most small institutions in Pakistan manage fees using Excel spreadsheets or paper registers. While Excel allows basic calculations, it does not enforce data integrity, cannot handle multiple users simultaneously, and becomes unmanageable at scale. Commercial ERP fee modules exist but are expensive and overly complex for small institutions. Our proposed SFMS is lightweight, focused specifically on fee management, free to deploy, and built on a normalized relational database — providing data accuracy, integrity enforcement, and reporting capabilities that neither Excel nor heavy ERP systems offer affordably.

12. Proposed Technology Stack

Frontend	Python — Tkinter (Desktop GUI)
Backend	Python 3.x
Database	MySQL (via MySQL Workbench)
Connectivity	mysql-connector-python library
Tools	VS Code, MySQL Workbench, draw.io (ER Diagram)

13. Expected Outcomes

Upon completion, the Student Fee Management System will:

- Provide a fully functional desktop application connected to a live MySQL database.
- Eliminate manual fee tracking errors through automated calculations and database constraints.
- Allow finance staff to record, search, and manage student payment records efficiently.
- Generate clear semester and department-wise reports for administrative decisions.
- Demonstrate a complete implementation of database design principles including normalization, entity relationships, and data integrity.